第26课 字符串（二）——常用操作

一、课程信息

项目	内容
课次	第 26 课 / 共 40 课
主题	字符串（二）——常用操作
前置知识	第25课：string基础（声明、输入输出、下标访问、`.size()` / `.length()`）
后续衔接	第27课：前缀和与差分入门

教学目标（可检验）

1. **能独立使用** `+`、`==`、`<`、`substr()`、`find()` 五种字符串操作完成编程任务。
2. **能用 for 循环遍历字符串**，完成"统计某字符出现次数""大小写转换""字符串翻转"三类典型问题。
3. **能说出** ASCII 码中 `'A'`、`'a'`、`'0'` 的值，并利用字符与整数的转换关系解题。

二、教学重点与难点

重点

- 字符串拼接（`+`）和比较（`==`、`<`、`>` 字典序）的含义与用法。
- `substr(pos, len)` 截取子串和 `find()` 查找子串。
- 用循环遍历字符串，逐个字符处理。

难点

- **字典序比较**的概念——学生容易与"长度比较"混淆。
- **`substr` 的下标从 0 开始**——越界、参数含义容易搞错。
- **ASCII 码**——第一次接触"字符就是数字"的概念，需要多举例过渡。

三、课堂时间分配（共 100 分钟）

阶段	时长	内容
复习热身	8 分钟	快问快答回顾第25课知识点，1道热身小题
讲解一：拼接与比较	15 分钟	讲故事引入拼接、字典序比较，配 2 个随堂演示
练习一	10 分钟	学生上机完成练习题 1
讲解二：substr 与 find	15 分钟	比喻讲解截取与查找，配 2 个随堂演示
练习二	10 分钟	学生上机完成练习题 2
讲解三：遍历字符串与 ASCII	20 分钟	ASCII 故事引入，统计字符、大小写转换、翻转，配 3 个演示
练习三	15 分钟	学生上机完成练习题 3、4
总结与作业	7 分钟	知识点回顾、布置课后作业、预告下节课

> 注：每段讲解不超过 20 分钟，讲练交替保持专注。

四、讲稿提纲

4.1 复习热身（8 分钟）

> 同学们好！上节课我们认识了一个新朋友——**string**。谁来回忆一下，string 就像什么？
> 对了，string 就像一列**小火车**，每节车厢里坐着一个字符，第一节车厢的编号是 **0**。
> 快问快答时间！
> - `string s = "hello";` 里 `s[0]` 是什么？—— `'h'`
> - `s.size()` 等于多少？—— `5`
> - 怎样用 `getline` 读一行带空格的话？—— `getline(cin, s);`
> 很好！今天我们要给这列小火车装上**新技能**：拼接车厢、比大小、截一段出来、找某节车厢在哪……

热身小题（口答）：声明一个字符串 `s`，读入后输出它的最后一个字符。

```
string s;
cin >> s;
cout << s[s.size() - 1];
```

4.2 讲解一：拼接与比较（15 分钟）

A. 字符串拼接：`+`

> 想象你有两列小火车，一列写着 "Hello"，一列写着 "World"。把它们**连在一起**，就变成 "HelloWorld"。在 C++ 里，用加号 `+` 就能做到！

```
string a = "Hello";
string b = "World";
string c = a + b;     // c 是 "HelloWorld"
string d = a + " " + b; // d 是 "Hello World"
```

> **注意：** 也可以拼一个字符上去哦！

```
string s = "abc";
s = s + 'd';   // s 变成 "abcd"
s += 'e';      // s 变成 "abcde"，更简洁的写法
```

随堂演示1： 读入姓和名，输出完整姓名。

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string xing, ming;
    cin >> xing >> ming;
    cout << xing + ming << endl;
    return 0;
}
```

B. 字符串比较：字典序

> 你们查字典的时候怎么比较两个单词谁在前面？先看第一个字母，a 在 b 前面；如果第一个字母一样，就看第二个……这就叫**字典序**。
>
> C++ 里比较字符串用的 `<`、`>`、`==` 就是按字典序来的！

```
string s1 = "apple", s2 = "banana";
cout << (s1 < s2) << endl;   // 输出 1（true），因为 a 在 b 前面
cout << (s1 == s2) << endl;  // 输出 0（false）
```

> **小陷阱：** `"apple" < "banana"` 是对的，但 `"9" > "10"` 也是对的！为什么？因为字典序逐个字符比，`'9' > '1'`，所以 "9" > "10"。这跟数字大小不一样哦！

随堂演示2： 读入两个单词，按字典序从小到大输出。

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string a, b;
    cin >> a >> b;
    if(a <= b)
        cout << a << " " << b << endl;
    else
        cout << b << " " << a << endl;
    return 0;
}
```

4.3 讲解二：substr 与 find（15 分钟）

A. 截取子串：`s.substr(pos, len)`

> 还记得我们的小火车吗？`substr` 就是从火车的**第 pos 节车厢开始**，往后**截 len 节**出来，组成一列新的小火车。
>
> 特别注意：pos 从 **0** 开始数！

```
string s = "HelloWorld";
// 下标:     0123456789
cout << s.substr(0, 5) << endl;  // 输出 "Hello"（从第0个开始，截5个）
cout << s.substr(5, 5) << endl;  // 输出 "World"（从第5个开始，截5个）
cout << s.substr(3, 4) << endl;  // 输出 "loWo"（从第3个开始，截4个）
```

> 如果只写一个参数 `s.substr(5)`，意思就是从第 5 个一直截到最后！

```
cout << s.substr(5) << endl;  // 输出 "World"
```

随堂演示3： 读入一个字符串和两个整数 pos、len，输出截取的子串。

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s;
    int pos, len;
    cin >> s >> pos >> len;
    cout << s.substr(pos, len) << endl;
    return 0;
}
```

B. 查找子串：`s.find("xxx")`

> `find` 就像在火车上**找人**。你告诉它你要找的内容，它告诉你**第一次出现的位置**（下标）。如果找不到，它会返回一个特殊的值 `string::npos`。

```
string s = "I love coding";
cout << s.find("love") << endl;    // 输出 2（从下标2开始出现）
cout << s.find("hate") << endl;    // 找不到，输出一个超大数字
```

> 判断找没找到的标准写法：

```
if(s.find("love") != string::npos){
    cout << "找到了! " << endl;
} else {
    cout << "没找到……" << endl;
}
```

随堂演示4： 读入一个句子和一个单词，判断句子中是否包含该单词。

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string sentence, word;
    getline(cin, sentence);
    cin >> word;
    if(sentence.find(word) != string::npos)
        cout << "Yes" << endl;
    else
        cout << "No" << endl;
    return 0;
}
```

4.4 讲解三：遍历字符串与 ASCII（20 分钟）

A. 用循环遍历字符串

> 我们的小火车有 `s.size()` 节车厢，编号从 0 到 `s.size()-1`。要**检查每节车厢**，就用 for 循环一节一节地走过去！

```
string s = "hello";
for(int i = 0; i < s.size(); i++){
    cout << s[i] << " ";
}
// 输出: h e l l o
```

B. ASCII 码——字符的"身份证号"

> 每个字符在计算机里都有一个**数字编号**，就像每个同学都有学号一样。这个编号系统叫 **ASCII 码**。
> 你们只需要记住三个"明星学号"：
> - **`'A'` = 65**，`'B'` = 66，……，`'Z'` = 90
> - **`'a'` = 97**，`'b'` = 98，……，`'z'` = 122
> - **`'0'` = 48**，`'1'` = 49，……，`'9'` = 57
>
> 发现了吗？大写字母和小写字母之间差了 **32**！`'a' - 'A' = 32`。
> 字符和整数可以互相转换，这是一个**超级有用**的技巧！

```
char c = 'A';
cout << (int)c << endl;      // 输出 65
cout << (char)(c + 32) << endl; // 输出 'a'（大写变小写）
```

> 判断一个字符是不是大写字母：

```
if(c >= 'A' && c <= 'Z'){
    cout << "是大写字母！" << endl;
}
```

C. 实用应用1：统计某字符出现次数

> 目标：数一数字符串里有多少个 `'a'`。

随堂演示5：

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s;
    cin >> s;
    int cnt = 0;
    for(int i = 0; i < s.size(); i++){
        if(s[i] == 'a') cnt++;
    }
    cout << cnt << endl;
    return 0;
}
```

D. 实用应用2：大小写转换

> 把一个全是小写字母的字符串变成全大写——每个字符减 32 就行！

随堂演示6：

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s;
    cin >> s;
    for(int i = 0; i < s.size(); i++){
        if(s[i] >= 'a' && s[i] <= 'z'){
            s[i] = s[i] - 32;  // 小写变大写
        }
    }
    cout << s << endl;
    return 0;
}
```

> 小技巧：也可以用 `toupper(s[i])` 和 `tolower(s[i])`，效果一样！

E. 实用应用3：字符串翻转

> 把 "hello" 变成 "olleh"——就像把小火车**掉个头**！
>
> 方法一：从后往前遍历，把字符拼到新字符串上。
> 方法二：首尾交换，就像排队时第一个和最后一个换位置，第二个和倒数第二个换位置……

随堂演示7（方法一）：

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s;
    cin >> s;
    string t = "";          // 准备一列空火车
    for(int i = s.size() - 1; i >= 0; i--){
        t += s[i];          // 从后往前，一节一节接上去
    }
    cout << t << endl;
    return 0;
}
```

随堂演示7（方法二，首尾交换）：

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    string s;
    cin >> s;
    int left = 0, right = s.size() - 1;
    while(left < right){
        swap(s[left], s[right]);
        left++;
        right--;
    }
    cout << s << endl;
    return 0;
}
```

> 彩蛋：其实 C++ 自带了一个 `reverse` 函数哦！`reverse(s.begin(), s.end());` 一行搞定！

七、常见错误与应对

序号	常见错误	典型表现	应对方法
1	**`substr` 下标从 1 开始**	写成 `s.substr(1, 3)` 以为从第 1 个字符开始	反复强调"下标从 0 开始"，黑板画火车图，让学生指着数
2	**字典序和数值序混淆**	认为 `"9" < "10"`	用"查字典"的比喻：只看第一个字母 `'9'` 和 `'1'`，`'9'` 排后面所以更大
3	**字符串比较用单等号**	写成 `if(s1 = s2)`	提醒：一个等号是"赋值"，两个等号才是"判断相等"——这是老朋友了！
4	**大小写转换忘记判断**	直接写 `s[i] -= 32`，导致非字母字符也被修改	强调必须先判断是不是字母，再做转换
5	**`find` 返回值直接当 bool**	写成 `if(s.find("abc"))`	解释：`find` 返回的是位置，找到位置 0 时条件反而为 false！必须和 `string::npos` 比较
6	**循环中 `i < s.size()` 类型警告**	`int i` 和 `size()` 返回的无符号类型比较	四年级阶段可以先忽略警告，后续课程再讲；也可写成 `i < (int)s.size()`
7	**凯撒加密不取余**	`'z' + 3` 变成 `'}'` 而不是 `'c'`	画一个圆形字母表，演示"到头了就绕回来"的感觉

九、下节课预告

> 同学们，今天我们给字符串小火车装上了好多新技能，大家表现得非常棒！
>
> 下节课我们要学习一个**全新的话题——前缀和与差分**。
>
> 想象一下：老师问你"从第 3 个同学到第 7 个同学的身高总和是多少"，你每次都要一个一个加太慢了。有没有一种**提前做好准备**的办法，让你一秒钟就能回答任意区间的总和呢？有的！那就是前缀和！
>
> 请大家提前复习一下**数组**和**循环**的知识，下节课我们就要用到啦！

教案编写完毕。祝教学顺利！